МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №8

з дисципліни «Функціональне програмування»

на тему: «Алгебраїчні типи даних “по-Python’івськи”»

Варіант 6: «Стани замовлення»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Опанувати концепцію алгебраїчних типів даних (ADT, sum types / tagged unions) у сучасній екосистемі Python; використати Union, Literal, typing.Protocol та структурний pattern matching (match/case) для моделювання станів та подій скінченного автомата переходу станів замовлення; забезпечити строгу перевірку типів через mypy --strict.

2. Теоретичні відомості

Алгебраїчні типи даних (ADT): типи-суми (Sum Types), що представляють значення, яке може набувати одного з декількох чітко визначених варіантів (варіантних класів). У Python ADT реалізуються через об'єднання незмінних dataclass (frozen=True) за допомогою TypeAlias та оператора | (Union).

Literal: тип, що обмежує множину допустимих значень суворо вказаними константами (наприклад, назвами подій переходів: 'pay' | 'ship' | 'deliver' | 'cancel').

typing.Protocol: структурний інтерфейс (duck typing на рівні статичного аналізатора), що описує спільні атрибути всіх станів (order_id, status_name, can_cancel, total_amount).

Структурний Pattern Matching (match/case): введений у Python 3.10 синтаксис, що дозволяє зіставляти складні форми об'єктів за типами та полями з автоматичною деструктуризацією, виступаючи чистою заміною каскадів isinstance та if/elif.

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Стани замовлення.

ADT / вузли: Created, Paid, Shipped, Delivered, Cancelled.

Основне завдання: Моделювати життєвий цикл замовлення як незмінний скінченний автомат. Реалізувати чисту функцію transition(state, event, **kwargs) через match/case, заборонивши некоректні переходи (з генерацією винятку InvalidStateTransitionError).

Базове завдання: Модель арифметичних виразів Expr = Num | Add | Mul | Sub | Div з рекурсивним обчисленням eval_expr та текстовим форматуванням pretty_expr.

4. Хід роботи та лістинг програми

ADT станів та контракт Protocol (model.py)

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Literal, Protocol, TypeAlias


class InvalidStateTransitionError(ValueError):
    pass


@dataclass(frozen=True, slots=True)
class OrderItem:
    name: str
    price: float
    qty: int

    @property
    def subtotal(self) -> float:
        return self.price * self.qty


@dataclass(frozen=True, slots=True)
class Created:
    order_id: int
    items: tuple[OrderItem, ...]
    created_at: str

    @property
    def status_name(self) -> str:
        return "Created"

    @property
    def can_cancel(self) -> bool:
        return True

    @property
    def total_amount(self) -> float:
        return sum(it.subtotal for it in self.items)


@dataclass(frozen=True, slots=True)
class Paid:
    order_id: int
    items: tuple[OrderItem, ...]
    transaction_id: str
    paid_at: str

    @property
    def status_name(self) -> str:
        return "Paid"

    @property
```

```
    def can_cancel(self) -> bool:
        return True

    @property
    def total_amount(self) -> float:
        return sum(it.subtotal for it in self.items)


@dataclass(frozen=True, slots=True)
class Shipped:
    order_id: int
    items: tuple[OrderItem, ...]
    tracking_code: str
    carrier: str
    shipped_at: str

    @property
    def status_name(self) -> str:
        return "Shipped"

    @property
    def can_cancel(self) -> bool:
        return False

    @property
    def total_amount(self) -> float:
        return sum(it.subtotal for it in self.items)


@dataclass(frozen=True, slots=True)
class Delivered:
    order_id: int
    items: tuple[OrderItem, ...]
    delivered_at: str
    signed_by: str

    @property
    def status_name(self) -> str:
        return "Delivered"

    @property
    def can_cancel(self) -> bool:
        return False

    @property
    def total_amount(self) -> float:
        return sum(it.subtotal for it in self.items)


@dataclass(frozen=True, slots=True)
class Cancelled:
    order_id: int
    items: tuple[OrderItem, ...]
    reason: str
    cancelled_at: str

    @property
    def status_name(self) -> str:
        return "Cancelled"

    @property
```

```
    def can_cancel(self) -> bool:
        return False

    @property
    def total_amount(self) -> float:
        return sum(it.subtotal for it in self.items)


OrderState: TypeAlias = Created | Paid | Shipped | Delivered | Cancelled

OrderEvent: TypeAlias = Literal["pay", "ship", "deliver", "cancel"]


class OrderStateProtocol(Protocol):
    order_id: int
    items: tuple[OrderItem, ...]
    status_name: str
    can_cancel: bool
    total_amount: float
```

Машина переходів станів та чек (evaluator.py)

```
from __future__ import annotations

from typing import Any

from model import (
    Cancelled,
    Created,
    Delivered,
    InvalidStateTransitionError,
    OrderEvent,
    OrderState,
    Paid,
    Shipped,
)


def transition(
    state: OrderState,
    event: OrderEvent,
    **kwargs: Any,
) -> OrderState:
    match (state, event):
        case (Created(order_id, items), "pay"):
            tx_id = str(kwargs.get("transaction_id", f"tx_{order_id}"))
            paid_at = str(kwargs.get("timestamp", "2026-11-11T12:00:00Z"))
            return Paid(
                order_id=order_id,
                items=items,
                transaction_id=tx_id,
                paid_at=paid_at,
            )

        case (Created(order_id, items), "cancel"):
            reason = str(kwargs.get("reason", "Cancelled before payment"))
            cancelled_at = str(kwargs.get("timestamp", "2026-11-11T12:00:00Z"))
            return Cancelled(
                order_id=order_id,
```

```
                items=items,
                reason=reason,
                cancelled_at=cancelled_at,
            )

        case (Paid(order_id, items), "ship"):
            tracking = str(kwargs.get("tracking_code", f"TRK{order_id:06d}"))
            carrier = str(kwargs.get("carrier", "Nova Poshta"))
            shipped_at = str(kwargs.get("timestamp", "2026-11-11T14:00:00Z"))
            return Shipped(
                order_id=order_id,
                items=items,
                tracking_code=tracking,
                carrier=carrier,
                shipped_at=shipped_at,
            )

        case (Paid(order_id, items), "cancel"):
            reason = str(kwargs.get("reason", "Refund requested"))
            cancelled_at = str(kwargs.get("timestamp", "2026-11-11T12:30:00Z"))
            return Cancelled(
                order_id=order_id,
                items=items,
                reason=reason,
                cancelled_at=cancelled_at,
            )

        case (Shipped(order_id, items), "deliver"):
            delivered_at = str(kwargs.get("timestamp", "2026-11-12T10:00:00Z"))
            signed = str(kwargs.get("signed_by", "Олексій"))
            return Delivered(
                order_id=order_id,
                items=items,
                delivered_at=delivered_at,
                signed_by=signed,
            )

        case _:
            raise InvalidStateTransitionError(
                f"Invalid transition from '{state.status_name}' with '{event}'"
            )


def progress_percentage(state: OrderState) -> float:
    match state:
        case Created():
            return 10.0
        case Paid():
            return 40.0
        case Shipped():
            return 75.0
        case Delivered():
            return 100.0
        case Cancelled():
            return 0.0


def render_receipt(state: OrderState) -> str:
    lines = [
        f"Замовлення #{state.order_id} | Статус: {state.status_name}",
        f"Прогрес виконання: {progress_percentage(state):.0f}%",
```

```
        "Товари:",
    ]
    for it in state.items:
        lines.append(f"  - {it.name} (x{it.qty}): {it.subtotal:.2f} грн")
    lines.append(f"Всього до сплати: {state.total_amount:.2f} грн")

    match state:
        case Paid(_, _, tx_id, p_at):
            lines.append(f"Оплата успішна: транзакція {tx_id} о {p_at}")
        case Shipped(_, _, trk, carrier, s_at):
            lines.append(f"Відправлено: {carrier}, ТТН {trk} о {s_at}")
        case Delivered(_, _, d_at, signed):
            lines.append(f"Доставлено о {d_at}, отримав: {signed}")
        case Cancelled(_, _, reason, c_at):
            lines.append(f"Скасовано о {c_at}. Причина: {reason}")
        case Created():
            lines.append("Очікує оплати.")

    return "\n".join(lines)
```

Базові вирази AST (Expr) (expr.py)

```
from __future__ import annotations

from dataclasses import dataclass
from typing import TypeAlias


@dataclass(frozen=True, slots=True)
class Num:
    val: float


@dataclass(frozen=True, slots=True)
class Add:
    left: Expr
    right: Expr


@dataclass(frozen=True, slots=True)
class Mul:
    left: Expr
    right: Expr


@dataclass(frozen=True, slots=True)
class Sub:
    left: Expr
    right: Expr


@dataclass(frozen=True, slots=True)
class Div:
    left: Expr
    right: Expr


Expr: TypeAlias = Num | Add | Mul | Sub | Div
```

Обчислювач виразів (eval_expr) (eval_.py)

```
from __future__ import annotations

from expr import Add, Div, Expr, Mul, Num, Sub


def eval_expr(expr: Expr) -> float:
    match expr:
        case Num(val):
            return float(val)
        case Add(left, right):
            return eval_expr(left) + eval_expr(right)
        case Sub(left, right):
            return eval_expr(left) - eval_expr(right)
        case Mul(left, right):
            return eval_expr(left) * eval_expr(right)
        case Div(left, right):
            denominator = eval_expr(right)
            if denominator == 0.0:
                raise ZeroDivisionError("Division by zero in Expr")
            return eval_expr(left) / denominator
```

Форматування виразів (pretty_expr) (pretty.py)

```
from __future__ import annotations

from expr import Add, Div, Expr, Mul, Num, Sub


def pretty_expr(expr: Expr) -> str:
    match expr:
        case Num(val):
            return f"{val:g}"
        case Add(left, right):
            return f"({pretty_expr(left)} + {pretty_expr(right)})"
        case Sub(left, right):
            return f"({pretty_expr(left)} - {pretty_expr(right)})"
        case Mul(left, right):
            return f"({pretty_expr(left)} * {pretty_expr(right)})"
        case Div(left, right):
            return f"({pretty_expr(left)} / {pretty_expr(right)})"
```

Демонстраційний модуль (main.py)

```
from __future__ import annotations

from eval_ import eval_expr
from evaluator import render_receipt, transition
from expr import Add, Mul, Num
from model import Created, OrderItem
from pretty import pretty_expr


def main() -> None:
```

```
    items = (
        OrderItem(name="Ноутбук", price=25000.0, qty=1),
        OrderItem(name="Мишка бездротова", price=850.0, qty=2),
    )

    state1 = Created(order_id=501, items=items, created_at="2026-11-11T10:00:00Z")
    print("=== ПОЧАТКОВИЙ СТАН ===")
    print(render_receipt(state1))

    state2 = transition(
        state1, "pay", transaction_id="TX_998877", timestamp="2026-11-11T10:15:00Z"
    )
    state3 = transition(
        state2, "ship", tracking_code="20450012345678", carrier="Нова Пошта"
    )
    state4 = transition(state3, "deliver", signed_by="Олексій Корж")

    print("\n=== ФІНАЛЬНИЙ СТАН ДОСТАВКИ ===")
    print(render_receipt(state4))

    math_tree = Mul(Add(Num(10), Num(5)), Num(2))
    print("\n=== АЛГЕБРАЇЧНІ ВИРАЗИ EXPR ===")
    print("Вираз:     ", pretty_expr(math_tree))
    print("Результат: ", eval_expr(math_tree))


if __name__ == "__main__":
    main()
```

Тести станів замовлення (tests/test_order_states.py)

```
import pytest
from evaluator import progress_percentage, render_receipt, transition
from model import (
    Cancelled,
    Created,
    Delivered,
    InvalidStateTransitionError,
    Paid,
    Shipped,
)


def test_full_successful_order_lifecycle(sample_created_order: Created) -> None:
    assert sample_created_order.can_cancel is True
    assert progress_percentage(sample_created_order) == 10.0

    paid = transition(sample_created_order, "pay", transaction_id="tx_123")
    assert isinstance(paid, Paid)
    assert paid.transaction_id == "tx_123"
    assert paid.can_cancel is True
    assert progress_percentage(paid) == 40.0

    shipped = transition(paid, "ship", tracking_code="TRK100", carrier="Meest")
    assert isinstance(shipped, Shipped)
    assert shipped.tracking_code == "TRK100"
    assert shipped.can_cancel is False
    assert progress_percentage(shipped) == 75.0
```

```
    delivered = transition(shipped, "deliver", signed_by="Олексій")
    assert isinstance(delivered, Delivered)
    assert delivered.signed_by == "Олексій"
    assert delivered.can_cancel is False
    assert progress_percentage(delivered) == 100.0


def test_cancellation_from_created(sample_created_order: Created) -> None:
    cancelled = transition(sample_created_order, "cancel", reason="Changed mind")
    assert isinstance(cancelled, Cancelled)
    assert cancelled.reason == "Changed mind"
    assert cancelled.can_cancel is False
    assert progress_percentage(cancelled) == 0.0


def test_cancellation_from_paid(sample_created_order: Created) -> None:
    paid = transition(sample_created_order, "pay", transaction_id="tx_99")
    cancelled = transition(paid, "cancel", reason="Refund requested")
    assert isinstance(cancelled, Cancelled)
    assert cancelled.reason == "Refund requested"


def test_invalid_transitions(sample_created_order: Created) -> None:
    with pytest.raises(InvalidStateTransitionError):
        transition(sample_created_order, "ship")

    with pytest.raises(InvalidStateTransitionError):
        transition(sample_created_order, "deliver")

    paid = transition(sample_created_order, "pay")
    with pytest.raises(InvalidStateTransitionError):
        transition(paid, "deliver")

    shipped = transition(paid, "ship")
    with pytest.raises(InvalidStateTransitionError):
        transition(shipped, "cancel")

    delivered = transition(shipped, "deliver")
    with pytest.raises(InvalidStateTransitionError):
        transition(delivered, "cancel")


def test_render_receipt(sample_created_order: Created) -> None:
    receipt = render_receipt(sample_created_order)
    assert "Замовлення #1" in receipt
    assert "Клавіатура" in receipt
    assert "1800.00" in receipt
```

Тести арифметичних виразів (tests/test_expr.py)

```
import pytest
from eval_ import eval_expr
from expr import Add, Div, Mul, Num, Sub
from pretty import pretty_expr


def test_eval_expr_basic() -> None:
    expr = Add(Num(5), Mul(Num(3), Num(4)))
    assert eval_expr(expr) == 17.0
```

```
    assert pretty_expr(expr) == "(5 + (3 * 4))"


def test_eval_expr_sub_div() -> None:
    expr = Div(Sub(Num(20), Num(5)), Num(3))
    assert eval_expr(expr) == 5.0
    assert pretty_expr(expr) == "((20 - 5) / 3)"


def test_zero_division() -> None:
    expr = Div(Num(10), Num(0))
    with pytest.raises(ZeroDivisionError):
        eval_expr(expr)
```

Property-based тести (Hypothesis) (tests/test_properties.py)

```
from eval_ import eval_expr
from expr import Add, Expr, Num, Sub
from hypothesis import given
from hypothesis import strategies as st
from model import Created, OrderItem


def expr_strategy() -> st.SearchStrategy[Expr]:
    leaf = st.builds(
        Num,
        st.floats(
            min_value=-1000.0,
            max_value=1000.0,
            allow_nan=False,
            allow_infinity=False,
        ),
    )
    return st.recursive(
        leaf,
        lambda children: st.builds(Add, children, children)
        | st.builds(Sub, children, children),
        max_leaves=10,
    )


@given(expr_strategy())
def test_property_expr_evaluation(e: Expr) -> None:
    res1 = eval_expr(e)
    res2 = eval_expr(e)
    assert res1 == res2


@given(
    st.lists(
        st.builds(
            OrderItem,
            name=st.text(min_size=1, max_size=10),
            price=st.floats(
                min_value=1.0,
                max_value=500.0,
                allow_nan=False,
                allow_infinity=False,
            ),
```

```
            qty=st.integers(min_value=1, max_value=5),
        ),
        min_size=1,
        max_size=5,
    )
)
def test_property_order_total(items: list[OrderItem]) -> None:
    order = Created(1, tuple(items), "2026-11-11T00:00:00Z")
    expected = sum(it.price * it.qty for it in items)
    assert abs(order.total_amount - expected) < 1e-4
```

5. Результати виконання та тестування

Запуск демонстраційної програми main.py

```
$ python main.py
=== ПОЧАТКОВИЙ СТАН ===
Замовлення #501 | Статус: Created
Прогрес виконання: 10%
Товари:
  - Ноутбук (x1): 25000.00 грн
  - Мишка бездротова (x2): 1700.00 грн
Всього до сплати: 26700.00 грн
Очікує оплати.

=== ФІНАЛЬНИЙ СТАН ДОСТАВКИ ===
Замовлення #501 | Статус: Delivered
Прогрес виконання: 100%
Товари:
  - Ноутбук (x1): 25000.00 грн
  - Мишка бездротова (x2): 1700.00 грн
Всього до сплати: 26700.00 грн
Доставлено о 2026-11-12T10:00:00Z, отримав: Олексій Корж

=== АЛГЕБРАЇЧНІ ВИРАЗИ EXPR ===
Вираз:      ((10 + 5) * 2)
Результат:  30.0
```

Результати виконання тестів pytest

```
$ pytest -q
..........                                                               [100%]
10 passed in 1.24s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 10 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
10 files would be left unchanged.
```

6. Висновки

У лабораторній роботі №8 продемонстровано реалізацію алгебраїчних типів даних у Python. Створено надійну модель життєвого циклу замовлення, де всі допустимі стани зафіксовані як незмінні dataclass, події обмежені типом Literal, а переходи валідуються чистим match/case без довгих ланцюгів isinstance. Код є абсолютно типобезпечним та проходить mypy --strict.